

KamaClaude 面试问题与参考答案

本文只覆盖简历中的 KamaClaude 项目。回答以当前仓库实现为准，建议面试时先给结论，再根据追问展开实现细节。

一、项目与整体架构

1. 请用两分钟介绍 KamaClaude，它和普通的大模型 API 项目有什么区别？

KamaClaude 是一个本地 Coding Agent Runtime，不只是封装一次模型调用。它采用 Core daemon 与 CLI/TUI 分离架构，通过 JSON-RPC over NDJSON 通信；Core 内部包含 AgentLoop、工具注册与权限审批、事件流、Session 持久化、上下文压缩、Git Checkpoint、增量验证、子 Agent、MCP 和评测 Harness。核心区别是它可以持续规划、执行工具、观察结果、自动验证和恢复会话，而不是输入 Prompt 后直接返回文本。

2. 从用户输入任务到工具结果返回模型，完整链路是什么？

TUI 将请求通过 IPC 发给 Core；SessionManager 先持久化用户消息并创建 run_id；AgentRunner 组装 Provider、ToolRegistry、PermissionManager、Compactor 和 VerificationController；AgentLoop 调用模型获得文本或 tool call；tool call 经参数校验和权限审批后执行，结果写回 ExecutionContext 和 Session，再进入下一轮模型调用。事件同时通过 EventBus 推给 TUI，并写入 events.jsonl。

3. 为什么采用 Core daemon 与 CLI/TUI 分离，而不是单进程？

Daemon 负责真实任务和状态，客户端只负责交互，因此 TUI 崩溃不会天然终止任务，也可以复用同一后端接入 CLI、TUI 或 Web。代价是必须处理 IPC 协议、连接生命周期、事件订阅、重连和进程退出，工程复杂度更高。

4. TUI 意外退出后，Agent 为什么还能继续？

Run 由 Core 中的 asyncio Task 执行，不属于 TUI 进程。运行事件和 Session 消息持续写盘，重新启动客户端后可通过工作目录列出 Session 并恢复；但如果当时正等待权限审批，客户端断连会拒绝该 Session 的待审批请求，避免 Future 永久挂起。

5. AgentRunner 和 AgentLoop 分别负责什么？

AgentRunner 是组装层，负责创建运行目录、上下文、工具、事件写入器、压缩器和验证控制器。AgentLoop 是执行层，只负责循环调用模型、执行工具、写回结果和判断终止。分离后 Loop 易于单测，Runner 也能被 Session、子 Agent 和 Harness 复用。

6. 为什么选择 JSON-RPC 2.0 + NDJSON？

JSON-RPC 提供 request、response、error 和 id 的统一契约；NDJSON 用换行完成流式分帧，适合在长连接中同时传递响应和事件。它比自定义文本协议更规范，又比引入完整 HTTP/WebSocket 服务更轻量。

7. EventBus、events.jsonl 和 Trace 有什么区别？

EventBus 是进程内实时分发机制；events.jsonl 是单个 Run 的业务事件审计文件；Trace 记录 IPC、LLM 等更底层的跨层时间线。三者分别解决实时解耦、运行回放和系统诊断问题。

8. 如何避免多个 Session 或 Run 发生状态串扰？

每个 Session 有独立目录、current_id、run_ids 和 asyncio.Lock；每个 Run 有独立 events 和 task 目录；工具绑定规范化后的 workspace_root。需要诚实说明：并行子 Agent 仍共享同一工作区，同时写同一文件时目前没有文件级冲突合并机制。

9. session_id、run_id、node_id 分别是什么？

session_id 表示可持续恢复的对话；run_id 表示 Session 中一次用户请求触发的执行；node_id 表示 thread.jsonl 中一条消息或 Compact 节点。一个 Session 包含多个 Run，一个 Run 又会产生多个消息节点。

10. 为什么工具要通过 ToolRegistry 注册？

Registry 统一提供工具 Schema、名称查找、白名单过滤和调用入口，模型只能看到被注册的工具。这样参数校验、权限审批、事件记录和错误格式可以统一实现，也便于接入 MCP 和子 Agent 工具。

二、Agent Loop 与工具调用

11. Plan、Act、Observe 在代码中分别对应什么？

Plan 是 `provider.chat`；Act 是逐个调用 `invoke_tool`；Observe 是把 assistant 的 `tool_use` 和工具的 `tool_result` 写入 `ExecutionContext`，供下一轮模型读取。

12. 一次返回多个 tool call 时如何执行？

当前实现按返回顺序串行执行。它牺牲部分延迟，但能保持确定性，并降低多个写工具并发修改同一工作区的冲突风险。

13. 工具失败后为什么不立即终止？

工具失败被转换为 `is_error=true` 的 `tool_result`，模型可以读取错误并更换参数或策略。只有 LLM 异常、取消、达到步数上限或验证门禁耗尽等终止条件才会结束 Run。

14. 如何防止模型无限循环？

`ExecutionContext` 有 `max_steps`；验证循环另有 `max_attempts` 和 `max_total_seconds`；Bash、Git 和验证命令也有单次超时。多层边界共同限制失控运行。

15. max_steps 到达后会发生什么？

Loop 将 Run 标记为 `exceeded_max_steps`。SessionManager 最终会把 one-shot Session 关闭，把 chat Session 置为 `waiting_for_input`；Run 的失败原因仍保留在事件和结果中。

16. LLM 异常、工具异常和取消如何处理？

LLM 异常会将 Run 标记为 `llm_error`；工具异常被包装成错误结果并允许继续；`CancelledError` 会标记取消并继续向上传播，由 `RunManager`、`SessionManager` 和事件层完成状态更新与清理。

17. 如何保证工具参数合法？

内建工具使用 `Pydantic` 参数模型或 `JSON Schema` 校验。校验或执行异常统一变成 `ToolResult` 错误，避免未校验参数直接进入文件系统或子进程。

18. 为什么不同工具需要不同权限策略？

读取和搜索通常是低风险操作；编辑、写文件和 `Bash` 可能改变代码或访问系统资源。按工具和参数分级可以减少无意义审批，同时保留写操作的控制边界。

19. 为什么 Always Allow 后访问工作区外仍会询问？

权限链把工作区外访问作为强制 `ASK`，优先级高于 `Session` 和持久化缓存。这样 `Always Allow` 只降低同类正常操作的交互成本，不能绕过目录边界保护。

20. MCP 工具如何接入？

`McpTool` 把 `MCP Server` 返回的 `Schema` 和调用接口包装成 `BaseTool`，并用 `server_name__tool_name` 前缀避免重名。连接不可用、远端错误和未知异常都会转换为统一 `ToolResult`。

三、Session、消息图与崩溃恢复

21. 为什么使用 id、parent_id 和 current_id？

`id` 唯一标识节点，`parent_id` 表示因果链，`current_id` 表示当前有效 `Head`。相比线性数组，它可以保留全部历史，又能支持 `Compact`、历史分支和异常落盘后的 `Head` 恢复。

22. 为什么不能直接把 JSONL 最后一行作为当前消息？

最后一行可能是尚未由 `meta` 确认的孤儿节点，也可能属于未来的另一条分支。`current_id` 才是当前 `Session` 已确认的逻辑 `Head`。

23. Session 恢复算法是什么？

先严格读取节点并校验重复 `id`、缺失父节点和环；再从 `meta.current_id` 开始按 `parent_id` 回溯，遇到最近 `Compact` 或链根停止，最后反转为正序消息。

24. 为什么遇到 Compact 可以停止？

`Compact` 保存的是截至旧 `Head` 的累积摘要，加载时会转换为 `summary` 消息和确认回复。它已经承担此前上下文，所以无需继续把更早消息送入模型。

25. Append-only 的优点和代价是什么？

优点是会覆盖历史、便于审计和分支，也减少原地修改损坏整个文件的风险。代价是文件持续增长，读取时要建立 id 索引，并需要 Compact 和后续归档策略。

26. 节点 fsync 后、current_id 更新前崩溃怎么办？

节点会成为 durable orphan tail。恢复时只在“它是 current_id 的唯一子节点，并且属于最后一个 run_id”时快进 current_id；不满足唯一性就不自动猜测。

27. 什么是 durable orphan tail？

它是已经完整写入并 fsync 到 thread.jsonl，但 meta 尚未推进 Head 的节点。通过 parent_id、run_id 和唯一候选约束可以安全识别。

28. 为什么 flush 后还要 fsync？

flush 只把 Python 缓冲区交给操作系统，掉电或硬退出时仍可能丢失；fsync 请求操作系统把文件数据同步到稳定存储。

29. meta.json 如何避免半写？

先写 meta.json.tmp，再用 replace 原子替换正式文件，可避免读取到半个 JSON。严格来说，当前实现尚未对临时文件和父目录额外 fsync，因此对突然断电的最强持久性仍有提升空间。

30. tool_use 已落盘但 tool_result 未落盘怎么办？

恢复时扫描当前有效链中的未配对 tool_use，并追加合成的错误 tool_result，使消息重新满足模型协议，避免重复执行未知状态的副作用工具。

31. JSONL 最后一行只写了一半怎么办？

启动恢复使用严格 JSON 解析，遇到非法尾行或破损图会把该 Session 记录为恢复错误并跳过，不让一个坏 Session 阻塞其他 Session。

32. 为什么按 parent_id 而不是文件顺序恢复？

文件顺序只是物理追加顺序，消息图可能包含孤儿尾节点和未来分支。parent_id + current_id 才定义当前逻辑对话链。

33. Compact 后旧消息还存在吗？

存在。Compact 只是追加新节点并改变 active history 的读取边界，不删除旧节点，因此仍可审计并为历史分支提供依据。

34. Compact 如何避免截断工具调用？

自动压缩只在本轮工具结果已经追加、Run 仍需继续时触发；摘要前会对超长 tool_result 做有界截断，但仍保留 tool_call/tool_result 的结构化文本。恢复逻辑还会修复未配对调用。

35. 七个故障注入阶段是什么？

分别是节点写前、节点写后、flush 后、fsync 后、meta 临时文件写后、replace 前和 replace 后，覆盖消息写入与 Head 推进的关键窗口。

36. 21/21 恢复和 84 节点零丢失如何测量？

基准对七个阶段各执行三轮真实 daemon 硬退出，共 21 次；恢复后检查已确认节点、父链、current_id 和可继续对话能力。84 是这些运行中被确认应持久存在的节点总数。

37. resume p50 和 p95 如何解释？

p50 是恢复耗时中位数，代表典型体验；p95 是第 95 百分位，反映尾部延迟。该结果是指定 Windows 环境和基准数据上的工程测量，不应表述为所有机器的固定性能。

四、代码搜索与文件安全

38. search_code 为什么能从 3437ms 降到 413ms？

优先使用 Ripgrep，并按 JSON 行流式消费结果；达到全局上限立即终止进程；Python 回退路径在读取前过滤目录、后缀、文件大小和 ignore 规则。优化减少了无效遍历、全量输出和大文件读取。

39. 为什么使用 Ripgrep JSON 流？

JSON 事件能稳定获得路径、行号、列号和文本，流式读取可以边搜索边计数，不必等全部输出进入内存。

40. 为什么达到 max_results 后终止子进程？

如果只截断返回值，Ripgrep 仍会继续扫描仓库并消耗 CPU 和磁盘。主动终止才能真正限制时间和资源。

41. 没有 Ripgrep 怎么办？

回退到 Python 搜索器，并尽量保持固定文本、正则、大小写、Glob、列号、忽略规则和全局上限的一致语义。

42. 为什么大文件必须在读取前判断？

读入后再判断已经产生内存和 I/O 成本。Python 后端先用目录项或 stat 获取大小，超过阈值直接跳过。

43. stat 后文件并发变大怎么办？

真正读取时只读取 `_MAX_FILE_BYTES + 1`，因此即使文件在检查后增长也不会无界进入内存。

44. 用户 ignore_files 如何生效？

配置规则是在内建忽略目录基础上追加，而不是覆盖默认值，因此既保留 `.git`、虚拟环境、构建目录等保护，也允许用户增加后缀或文件名规则。

45. 如何防止路径穿越？

WorkspaceBoundary 对路径做 resolve，并要求最终路径位于 workspace_root 下；显式路径还拒绝绝对路径和 `..` 逃逸。工具不会仅靠字符串前缀判断。

46. Gitignore、默认忽略和用户忽略如何配合？

Ripgrep 后端遵守 Gitignore；内建过滤提供与仓库配置无关的安全默认；用户规则继续追加。候选只要命中任一排除条件就不读取。

五、Git Checkpoint、回滚与会话分支

47. 为什么 Checkpoint 不直接 git commit？

自动提交会污染用户分支、提交历史和暂存区。Checkpoint 使用 Git tree 和内部 ref 保存状态，不创建用户可见 Commit，也不要求工作区干净。

48. Checkpoint 保存哪些状态？

保存 HEAD、真实 index_tree 和包含已跟踪修改及受预算保护的未跟踪文件的 worktree_tree，并记录 session_id、node_id、run_id 和类型等元数据。

49. 如何不修改用户暂存区生成 Worktree Tree？

创建临时 GIT_INDEX_FILE，从真实 index_tree 开始，向临时 Index 应用已跟踪修改和允许的未跟踪文件，再执行 write-tree，最后删除临时 Index。

50. 回滚为什么要校验 HEAD 和 expected_state？

preview 后用户可能继续修改仓库。HEAD 校验防止跨提交恢复，expected_state 是当前仓库状态令牌，可阻止把过期确认应用到新状态。

51. preview 后用户又修改文件还能直接回滚吗？

不能。状态令牌会改变，Rollback 会拒绝并要求重新 Preview，让用户看到最新的待丢弃变化。

52. Undo 如何实现？

应用目标 Checkpoint 前先为当前状态创建 kind=undo 的新 Checkpoint；回滚后返回 undo_checkpoint_id，因此可再次恢复到回滚前状态。

53. 为什么未跟踪文件太多时拒绝 Checkpoint？

临时 Git add 仍可能扫描和读取数据集、模型或 LFS 文件。实现设置文件数、单文件大小、总字节和命令输出上限，超限时宁可失败，也不让 Core 长时间失控。

54. git add 卡住问题如何改进？

Checkpoint 使用临时 Index，不碰真实暂存区；先只枚举路径和元数据，再做硬预算检查；明确写工具可传精确 paths，避免隐式扫描整个仓库；Git 子进程还有超时和输出上限。

55. 历史节点没有 Checkpoint 怎么分支？

沿该节点的父链向前寻找最近的 mutation Checkpoint，再检查从 Checkpoint 到目标节点之间是否存在未覆盖的写操作。只有能证明代码状态一致时才允许创建分支。

56. 为什么有时必须拒绝历史分支？

如果最近 Checkpoint 后存在无法重放或未被快照覆盖的写操作，恢复得到的代码就不一定对应目标消息节点。系统选择拒绝，而不是生成聊天历史与代码状态错位的分支。

57. Compact 会阻止寻找历史 Checkpoint 吗？

不会。分支场景读取父链时使用 `stop_at_compact=False`，因为旧节点仍保存在 append-only 图中。

58. 为什么新分支使用独立 Session 和 detached worktree？

独立 Session 提供新的 current_id 和后续消息链；独立 worktree 隔离代码修改；detached 模式避免自动创建或污染用户分支。

59. 如何保证分支的消息与代码对应？

Checkpoint 元数据同时绑定 session_id、node_id 和 run_id；分支先确定目标节点及安全 Checkpoint，再在新 worktree 中恢复 tree，并记录 fork 来源。

六、增量测试与自动修复

60. Session 创建时如何生成测试结构索引？

如果 workspace_root 本身是 Git 仓库根，SessionManager 会尽力调用 ProjectTestIndexManager.initialize。索引失败只跳过预热，不阻塞普通聊天；成功后把测试总数和 tree_id 摘要写入 Session。

61. 为什么索引与 Git Tree ID 绑定？

Tree ID 是文件内容快照。只有索引 tree 与当前 worktree tree 一致时，依赖关系和测试计数才可信，也能通过两个 Tree 的 Diff 增量更新。

62. 用户离线修改后旧索引怎么办？

Resume 和下一次选择测试时重新捕获当前 Tree，把旧索引 Tree 与当前 Tree 做 Diff；少量变化增量更新，结构变化或大范围变化执行全量重建。

63. 如何从 Git Diff 找到相关测试？

先获得任务 baseline_tree 与当前 tree 的变化文件，再从被修改源码模块沿 reverse_dependencies 向依赖方传播，收集具有静态 pytest node 的测试模块；最后用语义变化进一步缩小到测试节点。

64. 正向和反向依赖分别是什么？

PythonFileRecord 保存当前文件导入了哪些模块；reverse_dependencies 保存某模块被哪些模块导入。增量选测主要从变化模块沿反向边寻找受影响测试。

65. 为什么不能只靠文件名映射？

测试文件名与源码并不总是——对应，公共组件、间接导入和跨模块调用都会漏测。依赖图能覆盖传递影响，AST 符号再负责降低过度选择。

66. Python Parser 如何定位函数和类？

DiffAnalyzer 从零上下文 Patch 得到 ChangedRanges，PythonParser 分别解析新旧源码 AST，用节点的 lineno/end_lineno 映射到函数、方法和类，再输出统一 SemanticChangeSummary。

67. DiffAnalyzer、Parser、Unified Model、Classifier 各做什么？

DiffAnalyzer 负责语言无关的变化范围；语言 Parser 负责提取符号；统一模型屏蔽语言差异；Classifier 判断变化类型、风险以及是否可以安全缩小测试。

68. 为什么需要统一符号模型？

上层只依赖“文件、符号、变化类型、风险、是否可缩小”等稳定字段。以后新增 JS/TS 或 Go Parser 时，无需重写 VerificationManager 和选择流程。

69. 不同修改类型如何处理？

低风险函数体修改可以基于导入绑定和测试函数引用缩小；签名、导入、文件新增删除、解析失败等高风险变化保持文件级选择或全量回退。

70. 动态导入和 monkeypatch 有什么影响？

静态 AST 不能完全捕获运行时依赖，因此可能降低召回。当前策略通过保守传播、无法确认时文件级选择和全量回退优先保证召回率。

71. 哪些情况降级为全量测试？

没有相关测试、项目结构文件变化、索引错误、工作区在索引期间持续变化、变更比例过高或无法建立可信选择时都会 Full Fallback。

72. 失败用例召回率 100% 是什么意思？

在已有 mutation 基准中，全量测试发现的失败用例都被增量选择复现，即 `reproduced failures / full-suite failures = 100%`。它只说明该基准没有漏掉已知失败，不能证明任意项目绝不漏测。

73. 如何计算精度和过度选择率？

符号选择精度可定义为选中测试中真正受影响测试的比例；过度选择率为 `1 - precision`。还应同时报告测试数量减少率和耗时减少率。

74. 为什么召回率 100% 仍可能选中 2083/2084？

召回关注有没有漏掉失败测试，不关心多选了多少。依赖传播过于保守时可以保持高召回，但测试削减和时间收益很小。

75. AST 语义缩小如何降低过度选择?

它从修改范围识别具体符号，再结合测试模块的 import binding 和每个 pytest node 引用的局部名称，只选择引用受影响符号的测试节点。

76. AST 解析失败怎么办?

索引记录 parse_error，并把该变化视为不可安全缩小；选择器保留更宽范围或全量验证，而不是根据不完整语义激进删减。

77. TUI 中增量测试百分比如何计算?

索引保存 total_tests，选择结果保存 selected_test_count，比例为 `selected / total * 100%`；Full Fallback 等价于 100%，并同时展示 fallback_reason。

78. 如何扩展 JS/TS 或 Go?

实现对应语言 Parser 输出相同统一符号模型，并在项目检测阶段选择 Parser；依赖传播、风险分类、验证计划和报告层可继续复用。

七、VerificationManager 与自动测试循环

79. VerificationManager 和让 Agent 自己判断测试有什么区别?

模型判断不稳定且不可审计。VerificationManager 根据项目文件生成确定性计划，执行命令并输出统一报告；VerificationController 再把它变成完成前硬门禁。

80. 如何识别 pytest、Ruff 和 Mypy?

Detector 检查 pyproject.toml、配置文件和项目结构，生成带来源、工具、生态和命令的 VerificationPlan，计划可以先展示再执行。

81. 为什么要解析日志而不是发送完整日志?

原始日志可能很大、重复且格式噪声多。Parser 提取状态、文件、行列、规则、消息和摘要，减少 Token，并让 Agent 能按诊断项稳定修复。

82. 不同测试工具如何统一?

每个 Parser 处理自己的输出格式，但最终映射到统一 VerificationResult、Diagnostic 和 VerificationReport；上层只判断 passed、status 和 diagnostics。

83. 为什么需要 Completion Gate?

Prompt 只能建议，模型仍可能修改后直接宣布完成。required_on_write 模式记录 dirty 状态，end_turn 前若未验证会由 Runtime 自动注入 verify_project。

84. 验证失败后如何进入修复循环?

结构化报告作为 tool_result 写入上下文，dirty 保持为 true；模型读取诊断后继续编辑并再次验证，直到通过或达到边界。

85. 如何限制自动修复循环？

VerificationController 同时限制 max_attempts、max_total_seconds，并受 Agent max_steps 约束。耗尽后返回稳定失败原因，而不是无限重试。

86. 用户拒绝必要验证权限怎么办？

Controller 把 permission_denied 作为终止原因，required_on_write 下不能成功结束；suggest 模式只提示，不做硬门禁。

87. 为什么只读 Bash 不触发 Checkpoint？

`git status`、`git diff`、`dir` 等明确只读命令不会改变代码，频繁快照只会增加延迟。包含管道、重定向或组合符的命令不会被乐观判断为只读。

88. 如何区分测试超时与代码失败？

Runner 给每项检查设置超时，超时、非零退出、权限拒绝和解析失败使用不同 status/error_type；Controller 据此生成不同终止原因和摘要。

八、Evaluation Harness

89. Harness 和普通单元测试有什么区别？

单元测试验证组件逻辑；Harness 把“给 Agent 一个真实任务并由外部 Oracle 判定”作为实验单元，关注任务成功率、耗时、Token、工具调用和可复现证据。

90. 为什么绑定固定 Commit？

固定 base_ref 可以保证任务初始代码一致，否则仓库变化会让不同模型或不同轮次的结果不可比较。

91. 为什么使用 detached worktree？

每次评测拥有隔离的代码目录，不修改原工作区；detached 模式不污染用户分支，也方便任务结束后整体清理。

92. 为什么 Oracle 必须独立于 Agent？

Agent 的自述不可信，它可能漏跑测试或误判输出。Harness 在 Agent 结束后用预先声明的命令单独运行验证，形成独立判据。

93. 让 Agent 自己报告通过有什么问题？

会产生自评偏差：Agent 可能选择性执行测试、忽略失败、修改测试或仅根据命令文本判断。外部 Oracle 把执行者和裁判分离。

94. Harness 保存哪些证据？

保存 request、result、metrics、verification、agent.patch 和 runs/<run_id>/events.jsonl，可还原输入、行为、代码变化、验证和资源消耗。

95. 如何定义评测成功？

当前要求 Agent 状态成功且外部 VerificationReport 通过；任一失败都不能算任务成功。基础设施异常和超时单独分类。

96. p50 和 p95 为什么比平均值更有用？

p50 表示典型延迟，p95 暴露尾部慢任务；平均值容易被少量极端值拉动，也不能说明大多数用户体验。

97. 超时后如何清理？

Agent Run 被 `asyncio.wait_for` 取消，Bash 在 `CancelledError` 时终止进程树；Harness 的 `finally` 再强制移除 `worktree`，并记录 `cleanup_error`。

98. Worktree 清理失败为什么算基础设施错误？

残留目录会污染后续评测、占用磁盘并降低可复现性。即使 Oracle 已完成，执行环境未正确收尾也不能视为一次健康运行。

99. 为什么 Harness 不等于 OS Sandbox？

Worktree 只隔离 Git 工作区，不限制进程访问系统其他目录、网络、CPU 或内存。真正 Sandbox 还需要容器、受限用户、Job Object、namespace/seccomp 等系统能力。

100. Bash 能否被完全限制在工作目录？

不能。`cwd` 和 `WorkspaceBoundary` 能约束内建文件工具，`PermissionManager` 也会对明显的目录外访问强制询问，但 Shell 本身仍具有系统权限，因此当前不能宣称强隔离。

九、子 Agent 与 Run 管理

101. 为什么子 Agent 使用冷启动上下文？

子 Agent 只接收父 Agent 显式提供的 Prompt，避免复制全部历史导致 Token 膨胀和无关信息干扰，也使任务边界更清晰。

102. 父 Agent 会保存子 Agent 的完整消息吗？

不会直接把子 Agent 的内部对话并入父 Session。前台模式只把最终结果作为 `spawn_agent` 的 `tool_result` 返回；后台模式返回 `run_id`，之后用 `agent_result` 获取最终结果。子 Run 自己保存 `events.jsonl`，事件会桥接给父 TUI 展示。

103. 为什么桥接子 Agent 事件？

父 TUI 可以看到子任务开始、工具调用、进度和结束，而不需要直接订阅每个子 EventBus；同时仍保留独立 `run_id`。

104. 前台和后台子 Agent 有什么区别？

前台调用等待子任务完成并立即返回结果；后台调用立即返回 run_id，任务由 BackgroundTaskRegistry 管理，父 Agent 可以继续工作后再查询。

105. 后台结果如何查询？

`agent_result(run_id)` 从注册表读取 Task 和 ExecutionContext；未完成返回 still running，完成后返回结果，取消或异常返回结构化错误。

106. 为什么限制嵌套深度？

无限派生会导致 Token、工具调用和并发数量失控。当前根 Agent 最多再形成有限层级，达到深度直接返回错误。

107. 父子 Agent 是否共享工作区和权限？

共享 workspace_root 和 PermissionManager，但各自有独立 ExecutionContext、run_id、EventBus 和事件文件。共享工作区便于协作，也意味着并行写冲突仍需额外控制。

108. 两个子 Agent 同时编辑同一文件怎么办？

当前没有自动三方合并或文件锁，因此存在最后写入覆盖和语义冲突风险。实际使用应把子任务按文件边界拆分，未来可加入写集合声明、文件锁或独立 worktree 合并。

109. 为什么取消父 Run 要级联取消子 Run？

RunManager 保存 parent_run_id 关系，级联取消从后代到父任务调用 Task.cancel，避免父任务结束后子进程继续占用服务器资源或修改代码。

110. 为什么慢任务只提醒而不会自动终止？

测试、构建和大仓库 Git 操作可能合法地耗时较长。监控依据总时长和无进度时长提示用户，再由用户取消或 snooze，可以避免错误终止有效工作。

111. Task.cancel 后 Bash/Git 子进程一定退出吗？

BashTool 捕获 CancelledError：Windows 使用 `taskkill /T /F`，Unix 使用进程组 SIGTERM，超时后再强杀。这样能覆盖常见子进程树，但操作系统级不可杀进程仍需由超时和关机日志兜底。

112. Core shutdown 为什么要设置最大等待时间？

取消是协作式的，某些任务可能卡在第三方库或系统调用。RunManager 先取消全部 Run，再有界等待；超时后记录仍未结束的任务，避免 daemon 永久停在 shutdown。

十、最优先准备的 15 个问题

建议优先熟练复述：1、2、3、21、23、26、30、38、47、55、63、72、79、89、109。

每题推荐采用以下结构：

1. 先用一句话说明要解决的问题。
2. 再说核心数据结构或执行流程。
3. 给出故障场景、基准数据或测试证据。

4. 最后主动说明当前边界和下一步改进。

十一、面试时不要过度承诺的内容

- 召回率 100% 只适用于当前 mutation 基准，不代表任意项目绝不漏测。
- Git worktree 和目录边界不是 OS Sandbox，Bash 仍具有宿主进程权限。
- Append-only + fsync 已覆盖进程硬退出，但 meta 文件和目录的掉电级持久性仍可继续加强。
- 并行子 Agent 共享工作区，目前没有自动文件冲突合并。
- Python AST 增量测试已实现；JS/TS、Go 是架构上的扩展方向，不应描述为已完成。
- p50/p95 必须同时说明测试环境、样本量和计时边界，不能只背数字。

十二、代码入口

- `src/kama_claude/core/loop.py`：Agent Loop 与验证门禁
- `src/kama_claude/core/runner.py`：运行时依赖组装和工具注册
- `src/kama_claude/core/session/store.py`：消息图、fsync、Compact 与恢复
- `src/kama_claude/core/session/manager.py`：Session 生命周期、Resume 和 Branch
- `src/kama_claude/core/tools/builtin/search_code.py`：流式代码搜索
- `src/kama_claude/core/git/checkpoint.py`：Git Tree Checkpoint、Rollback 和 Undo
- `src/kama_claude/core/verification/test_index.py`：AST 测试索引和增量选测
- `src/kama_claude/core/verification/controller.py`：自动验证与有界修复
- `src/kama_claude/core/harness/evaluation.py`：Evaluation Harness
- `src/kama_claude/core/subagent/tool.py`：子 Agent 生命周期
- `src/kama_claude/core/run_manager.py`：Run 监控和级联取消